



Ansible Secrets:

- Ansible may need access to sensitive data such as passwords or API keys in order to configure managed hosts.
- Normally, this information might be stored as plain text in inventory variables or other Ansible files. In that case, however, any user with access to the Ansible files or a version control system which stores the Ansible files would have access to this sensitive data. This poses an obvious security risk.
- Ansible **'Vault'**, which is included with Ansible, can be used to encrypt and decrypt any structured data file used by Ansible.
- To use Ansible Vault, a **command-line** tool named **'ansible-vault'** is used to create, edit, encrypt, decrypt, and view files.
- **'Ansible Vault'** can encrypt any structured data file used by Ansible. This might include inventory variables, included variable files in a playbook, variable files passed as arguments when executing the playbook, or variables defined in Ansible roles.

1: Creating an Encrypted File:

- The command prompts for the new vault password and then opens a file using the default editor, `'vi.'`
- You can set and export the EDITOR environment variable to specify a different default editor by setting and exporting. For example, to set the default editor to `'nano'`, `'export EDITOR=nano'`.

```
```bash
ansible-vault create secret.yml
```
```

- Instead of entering the vault password through standard input, you can use a vault password file to store the vault password.
- You need to carefully protect this file using file permissions and other means.

```
```bash
ansible-vault create --vault-password-file=vault-pass secret.yml
```
```

2: Viewing an Encrypted File:

- You can use the `ansible-vault view filename` command to view an Ansible Vault- encrypted file without opening it for editing.

```
```bash
ansible-vault view secret1.yml
```
```

3: Editing an Existing Encrypted File:

- To edit an existing encrypted file, Ansible Vault provides the `ansible-vault edit filename` command.
- This command decrypts the file to a temporary file and allows you to edit it.
- -When saved, it copies the content and removes the temporary file

```
```bash
ansible-vault edit secret.yml
```
```

REMEMBER THAT:

The `edit` subcommand always rewrites the file, so you should only use it when making changes. This can have implications when the file is kept under version

control. You should always use the view subcommand to view the file's contents without making changes

4:Encrypting an Existing File:

- To encrypt a file that already exists, use the `ansible-vault encrypt filename` command.
- This command can take the names of multiple files to be encrypted as arguments.

```
```bash
ansible-vault encrypt secret1.yml secret2.yml
```
```

- Use the `--output=OUTPUT\_FILE` option to save the encrypted file with a new name. You can only use one input file with the `--output` option

5: Decrypting an Existing File:

- An existing encrypted file can be permanently decrypted by using the `ansible-vault decrypt filename` command.
- When decrypting a single file, you can use the `--output` option to save the decrypted file under a different name.

```
```bashansible-vault decrypt secret1.yml --output=decrypted-1.yml```
```

## 6: Changing the Password of an Encrypted File:

- You can use the `ansible-vault rekey filename` command to change the password of an encrypted file.
- This command can rekey multiple data files at once. It prompts for the original password and then the new password.

```
```bash
ansible-vault rekey secret.yml
```
```

- When using a vault password file, use the `--new-vault-password-file` option:

```
```bash
ansible-vault rekey --new-vault-password-file=NEW_VAULT_PASSWORD_FILE secret.yml
```
```

## PLAYBOOKS AND ANSIBLE VAULT:

- To run a playbook that accesses files encrypted with Ansible Vault, you need to provide the encryption password to the ansible-playbook command.
- If you do not provide the password, the playbook returns an error:

```
```bash
[root@asnsiblu ~]# ansible-playbook site.yml

ERROR: A vault password must be specified to decrypt vars/api_key.yml
```
```

- To provide the vault password to the playbook, use the `--vault-id` option.
- For example, to provide the vault password interactively, use `--vault-id @prompt` as illustrated in the following

### example:

```
```bash
[root@asnsiblu ~]# ansible-playbook --vault-id @prompt site.yml

Vault password (default): redhat
```
```

### Note that:

- if you are using a release of Ansible earlier than version 2.4, you need to use the `--ask-vault-pass` option to interactively provide the vault password.
- you can still use this option if all vault-encrypted files used by the playbook were encrypted with the same password.

```
```bash
```

```
[root@asnsiblu ~]# ansible-playbook --ask-vault-pass site.yml
```

```
Vault password: redhat
```

```
...
```

- Alternatively, you can use the `--vault-password-file` option to specify a file that stores the encryption password in plain text.
- The password should be a string stored as a single line in the file.
- you can use multiple Ansible Vault passwords with `ansible-playbook`.
- To use multiple passwords, pass multiple `--vault-id` or `-- vault-password-file` options to the `ansible-playbook` command.

```
```bash
```

```
ansible-playbook --vault-id one@prompt --vault-id two@prompt site.yml
```

```
...
```

- Playbook variables (as opposed to inventory variables) can also be protected with Ansible Vault.
- Sensitive playbook variables can be placed in a separate file which is encrypted with `'Ansible Vault'` and which is included in the playbook through a `'vars_files` directive.'
- This can be useful, because playbook variables take precedence over inventory variables.
- If you are using multiple vault passwords with your playbook, make sure that each encrypted file is assigned a vault ID, and that you enter the matching password with that vault ID when running the playbook.
- This ensures that the correct password is selected first when decrypting the `'vault-encrypted'` file, which is faster than forcing Ansible to try all the vault passwords you provided until it finds the right one.

```
```bash
```

```
ansible-vault create --vault-id dev@prompt dev-secrets.yml
```

```
ansible-vault create --vault-id prod@prompt prod-secrets.yml
```

```
...
```

run playbook

```
'''
```

```
ansible-playbook site.yml --vault-id dev@prompt --vault-id prod@prompt
```

```
'''
```

Recommended Practices for Variable File Management:

- To simplify management, it makes sense to set up your Ansible project so that sensitive variables and all other variables are kept in separate files.
- The files containing sensitive variables can then be protected with the `ansible-vault` command.

